P04

GIT (II)

CONCEPTOS BÁSICOS DE GIT

- **Bifurcación (fork)**: copia de un repositorio que se crea en una cuenta diferente para desarrollar cambios sin afectar el original.

- **Rama (branch)**: Una rama es una versión paralela de un repositorio. Se incluye en del repositorio, pero no afecta a la rama primaria o principal, lo que te permite trabajar libremente sin interrumpir la versión "activa".

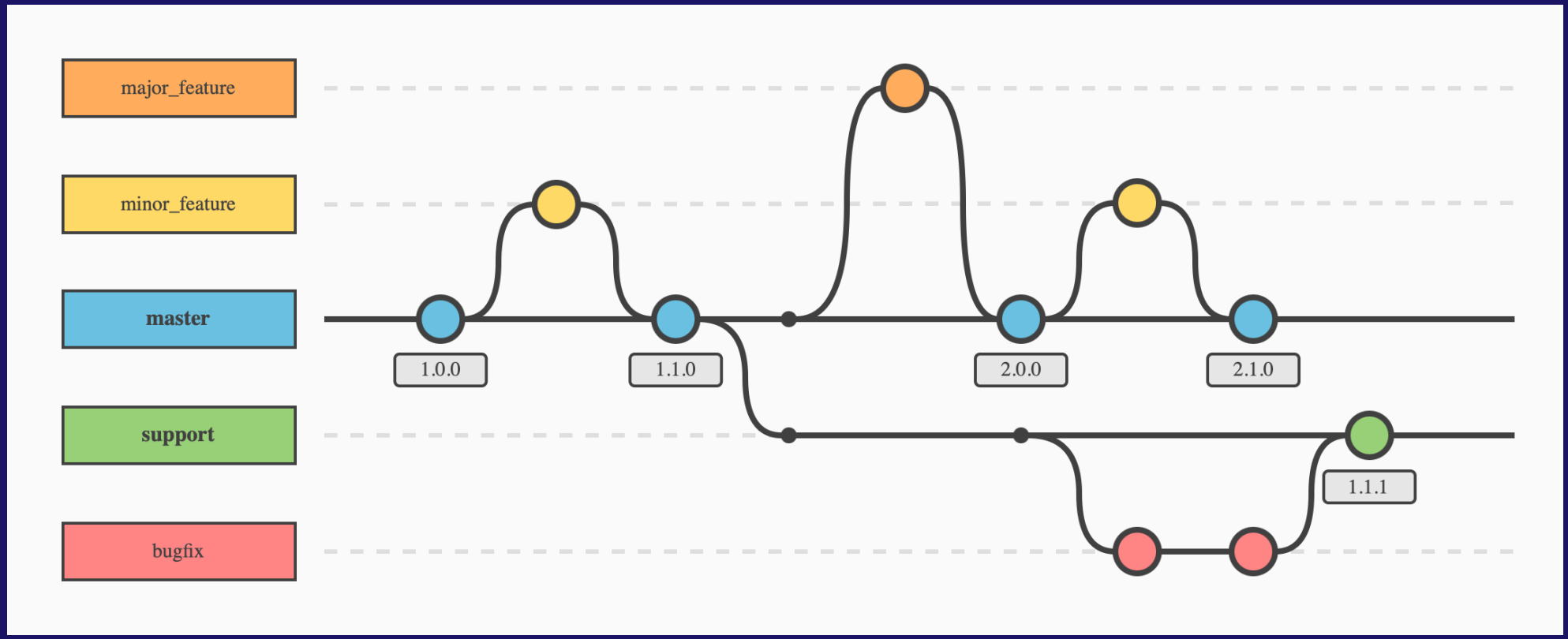
- **Cambiar de rama (checkout):** Cambia la rama actual

- **Mezclar (merge):** Es una forma eficaz de integrar los cambios de ramas divergentes.

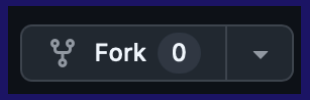
- **Solicitud de incorporación de cambios (pull request):** Es la acción de validar un código que se va a mezclar de una rama a otra. Una vez que hayamos realizado los cambios deseados, podemos volver a combinar una rama en la rama principal para publicarlos.

TAREA 3: TRABAJO CON RAMAS

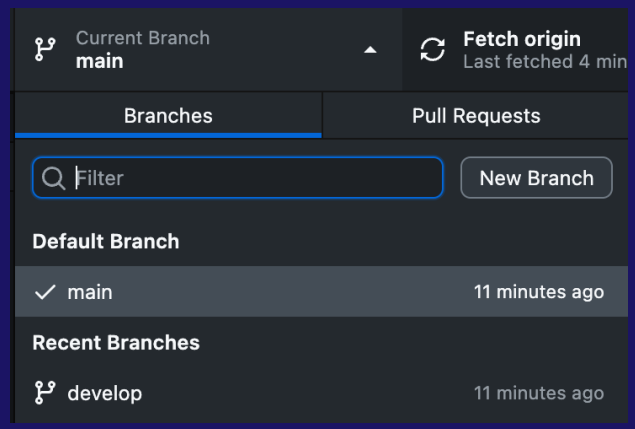
RAMAS

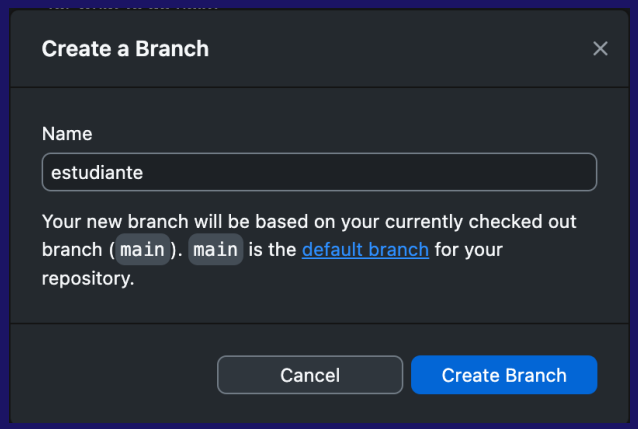

major_feature
minor_feature
master
support
bugfix
1.0.0
1.1.0
2.0.0
2.1.0
1.1.1

FORK



1. El coordinador del grupo crea un fork del repositorio en la web de GitHub: https://github.com/materuel-ua/prog2-fork.git
2. El coordinador del grupo invita al resto de integrantes
3. El resto de integrantes aceptan la invitación y clonan el repositorio en GitHub Desktop. Deben responder en GitHub Desktop: "For my own purposes"
4. Cada miembro del equipo creará una rama en GitHub Desktop para implementar métodos de las clases. Las ramas se llamarán:
   a. estudiante
   b. profesor
   c. curso
   d. horario
   e. aula





CLASE ESTUDIANTE

- **Atributos de clase:** total_estudiantes (contador de instancias de Estudiante).
- **Atributos de instancia:** nombre: str, edad: int, cursos_inscritos: List[str].
- **Métodos:**
    1. inscribir_curso(curso: str)
    2. mostrar_informacion()
    3. __str__()
    4. __repr__()
    5. desde_tupla(cls, tupla) (classmethod)
    6. es_mayor_de_edad() (estático)

CLASE PROFESOR

- **Atributos de clase:** total_profesores (contador de instancias de Profesor).
- **Atributos de instancia:** nombre: str, especialidad: str, cursos_asignados: List[str].
- **Métodos:**
    1. asignar_curso(curso: str)
    2. mostrar_informacion()
    3. __str__()
    4. __repr__()
    5. desde_tupla(cls, tupla) (classmethod)
    6. esta_disponible_para_nuevo_curso() (estático)

CLASE CURSO

- **Atributos de clase:** total_cursos (contador de instancias de Curso).
- **Atributos de instancia:** nombre: str, codigo: str, descripcion: str.
- **Métodos:**
    1. mostrar_detalles()
    2. actualizar_descripcion(nueva_descripcion: str)
    3. __str__()
    4. __repr__()
    5. desde_tupla(cls, tupla) (classmethod)
    6. es_curso_abierto() (estático)

CLASE HORARIO

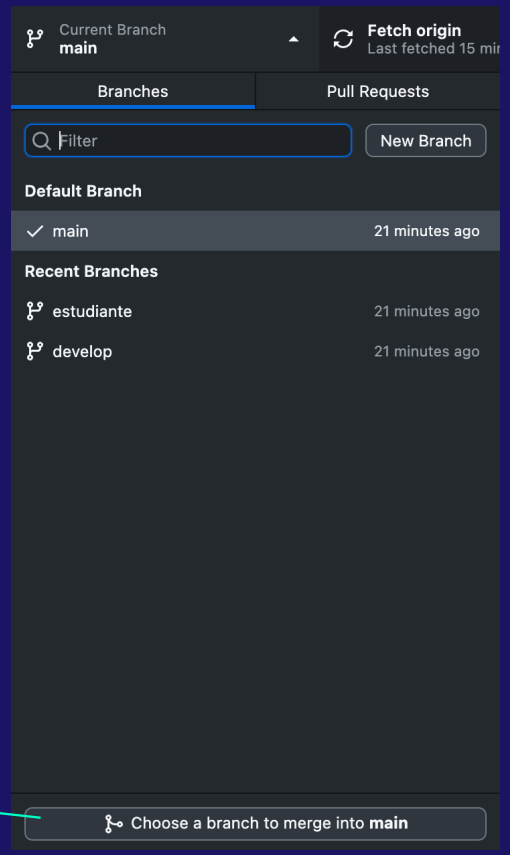
- **Atributos de clase:** total_horarios (contador de instancias de Horario).
- **Atributos de instancia:** curso: str, día: str, hora_inicio: str, hora_fin: str.
- **Métodos:**
    1. mostrar_horario()
    2. modificar_horario(nuevo_horario: dict)
    3. __str__()
    4. __repr__()
    5. desde_tupla(cls, tupla) (classmethod)
    6. es_horario_valido() (estático)

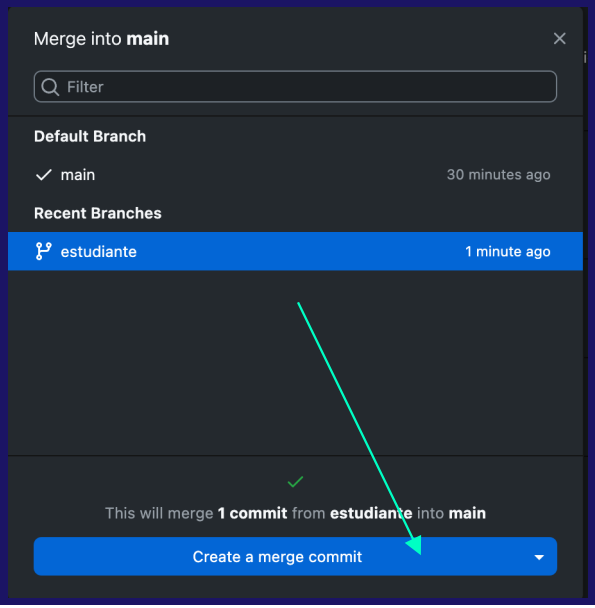
CLASE AULA

- **Atributos de clase:** total_aulas (contador de instancias de Aula).
- **Atributos de instancia:** numero: str, capacidad: int, recursos: List[str].
- **Métodos:**
    1. mostrar_detalles()
    2. reservar_aula()
    3. __str__()
    4. __repr__()
    5. desde_tupla(cls, tupla) (classmethod)
    6. es_adecuada_para_clase() (estático)

MERGE

1. Cuando cada integrante acabe, mezclará su rama con la rama main.
2. Para ello, hacemos checkout de la rama main primero
3. Seleccionamos la rama que mezclar con main
4. Mezclamos
5. Hacemos push
6. Los demás hacen pull





TAREA 4: PROTECCIÓN DE RAMAS

RAMA MAIN

Trabajar directamente sobre la rama main en Git es una mala práctica por varias razones:

- **Riesgo de errores en producción**: Si main es la rama estable y la usamos para desarrollo, cualquier error accidental puede afectar directamente al código en producción.
- **Dificulta la colaboración**: En equipos, varios desarrolladores trabajan en paralelo. Si todos modifican main, los conflictos de fusión serán más frecuentes y difíciles de manejar.
- **Historial de cambios desordenado**: Sin ramas de características (feature branches), el historial de main se llena de commits sin contexto claro, dificultando la depuración.
- **No permite revisiones de código efectivas**: Si trabajamos en ramas separadas, podemos hacer pull requests para revisión antes de integrar cambios en main.
- **Dificulta la reversión de cambios**: Si cometemos un error en main, revertirlo puede ser más problemático que si estuviera en una rama de desarrollo.

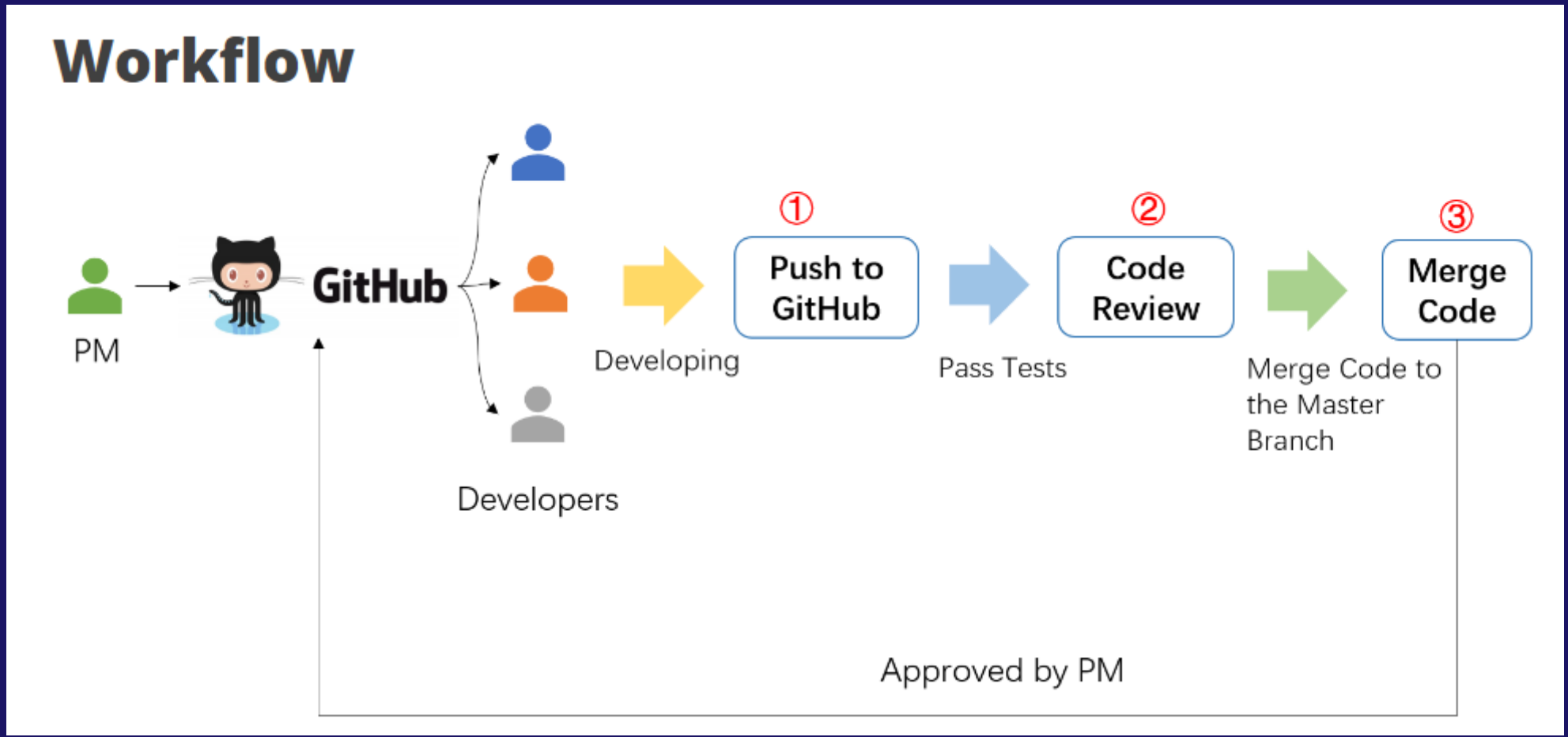
RAMA MAIN

Mejor práctica: usar ramas separadas

- **main**: Rama estable y lista para producción
- **develop**: Para integrar nuevas funciones antes de fusionarlas con main
- **feature/xyz**: Para desarrollar nuevas características sin afectar main

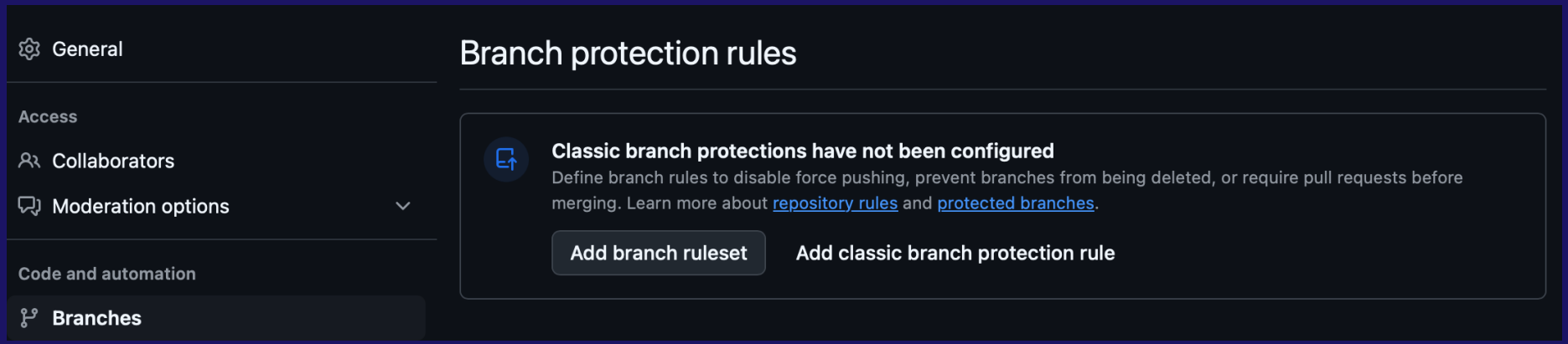
Trabajaremos en ramas separadas y solo fusionaremos con main cuando el código esté probado y revisado.

PULL REQUEST



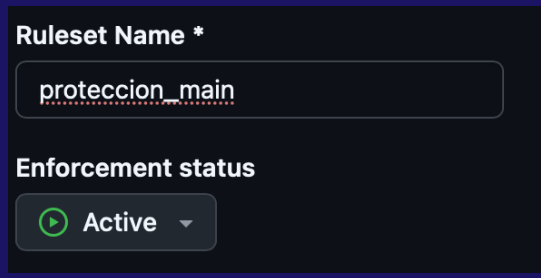
PROTEGIENDO LA RAMA MAIN

- El coordinador del grupo, irá a *Settings* → *Branches* → *Add branch ruleset*, en la Web del repositorio en GitHub



PROTEGIENDO LA RAMA MAIN

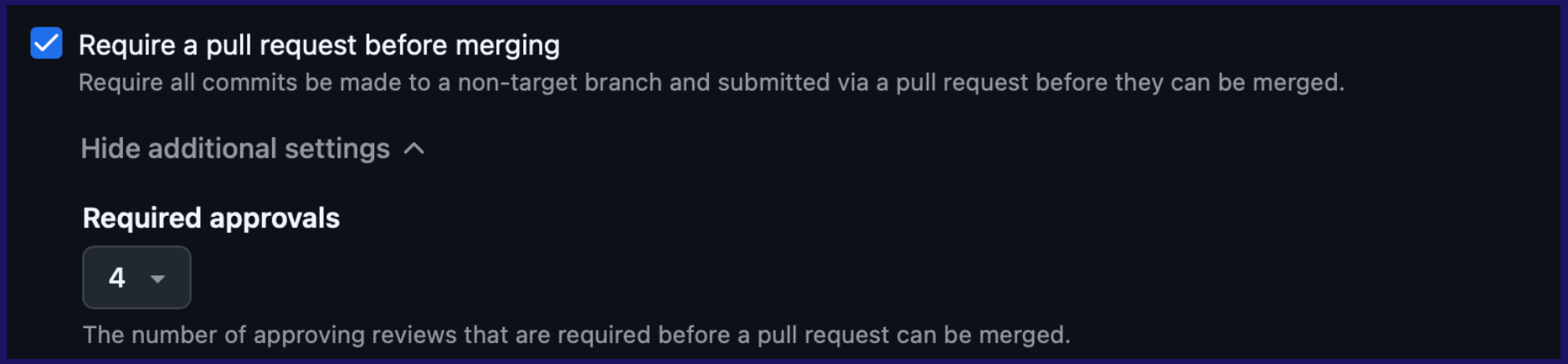
- Crearemos una regla llamada proteccion_main y la habilitaremos:
  Enforcement status → Active

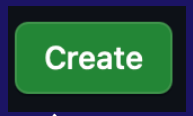


- Indicaremos en *Target Branches* que esta regla se aplique a la rama por defecto (main):

PROTEGIENDO LA RAMA MAIN

- Crearemos una regla de rama para requerir un pull request antes de mezclar. Requerirá de (n-1) aprobaciones, siendo n el nº de componentes del grupo.

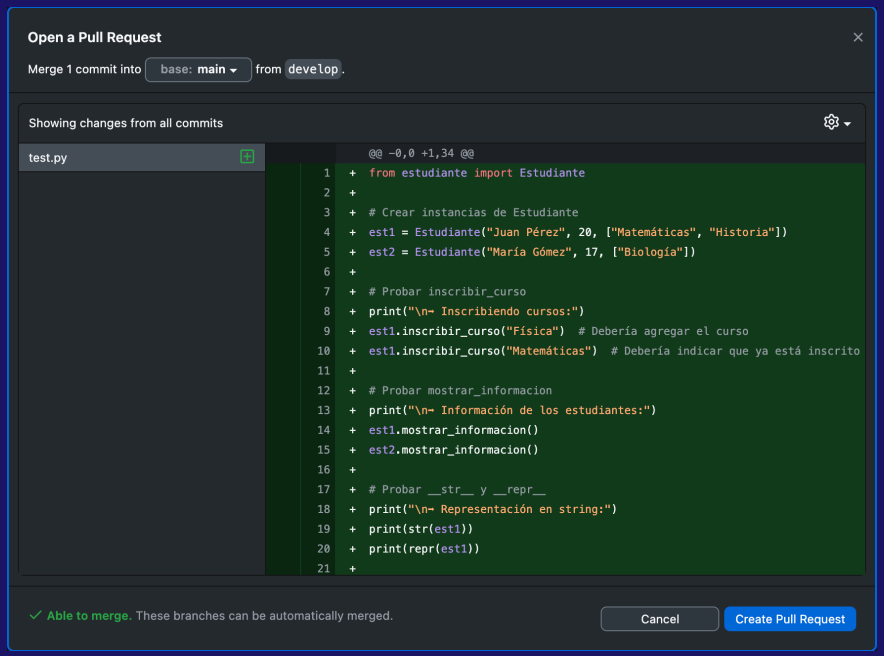


- Pulsamos 

para crear la regla. A partir de ahora, solo podemos escribir en main a través de un pull request. Veamos como se hace...

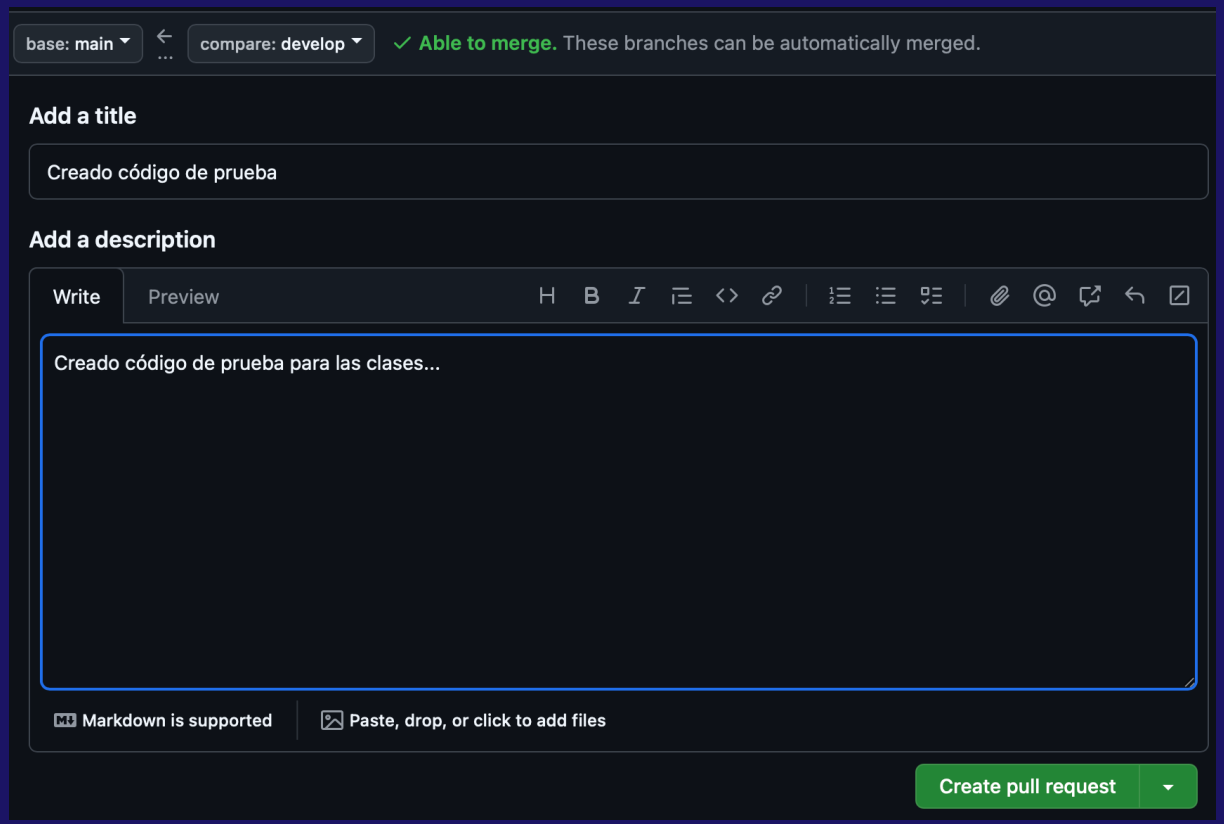
TAREA

- El coordinador del grupo crea en la rama develop (que tendrá que crear previamente) un fichero *test.py* para poder usar las clases que habéis creado anteriormente.
- Añade código para crear y probar objetos de las distintas clases.
- Hará commit y push.
- Vamos a previsualizar el pull request con *Branch* → *Preview Pull Request*
- Si es correcto, pulsaremos Create Pull Request. Aseguraos que estáis mezclando develop en main.



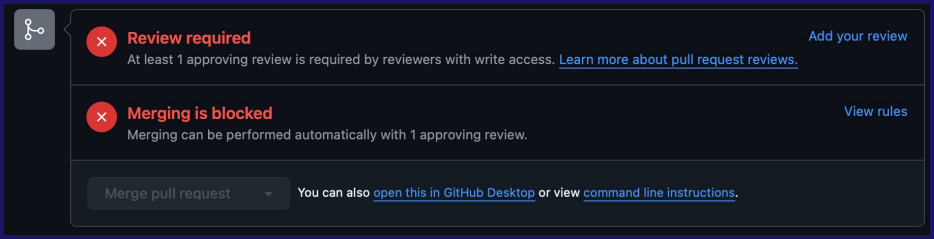
TAREA

- Se nos abrirá la Web de GitHub en la que tendremos que dar un título y una descripción al pull request.
- Tras ello, podremos crearlo

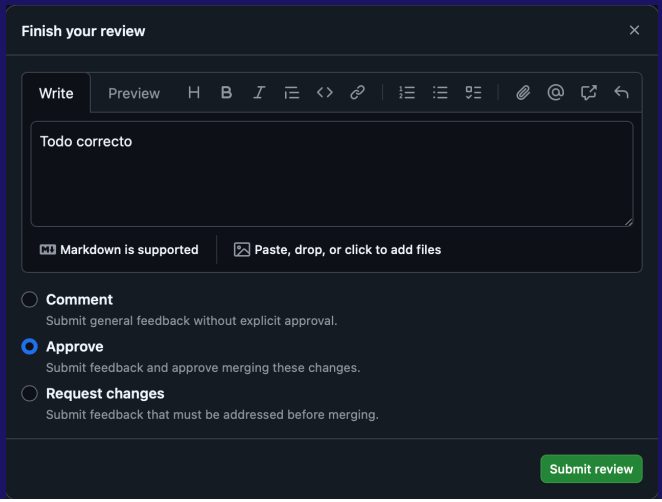


TAREA

- Si hemos protegido bien la rama main, no podremos mezclar las ramas hasta que (n-1) miembros del equipo hayan dado su autorización:

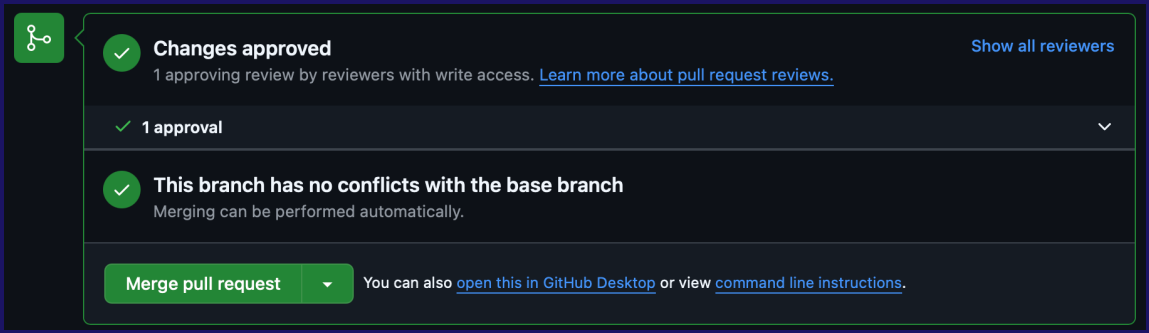


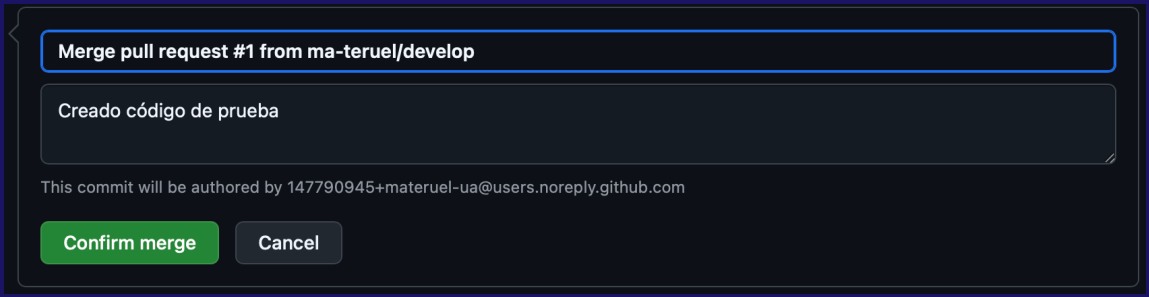
1. Cada miembro del equipo (a excepción de quien ha solicitado la pull request), irá a *Files changed*, analizará el código del compañero, y si es correcto, aprobará la mezcla de ramas

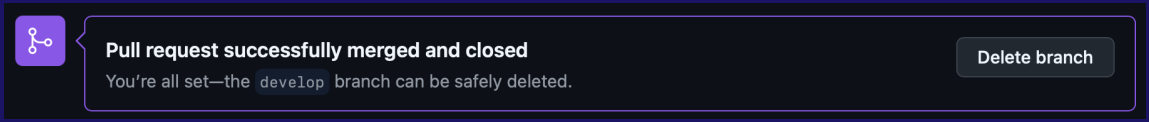


TAREA

- Cuando todos los compañeros hayan dado su aprobación, se podrán mezclar las ramas si no hay conflictos con el código:







CREANDO EL REPOSITORIO DEL PROYECTO GRUPAL

VUESTRO REPOSITORIO

1. Cread un repositorio en GitHub (público, con README.MD y .gitignore de Python) con el nombre **prog2-año-grupo**
   a. Siendo año los dos últimos dígitos del año actual
   b. Siendo grupo vuestro grupo de prácticas
   c. Por ejemplo: prog2-25-A1

2. Invitad al repositorio a todos los integrantes del grupo y a vuestro profesor de prácticas (ccacheroc o materuel-ua)

3. Debéis editar el fichero README.MD, usando para ello la plantilla disponible aquí (adaptando su contenido a vuestro trabajo)

4. Cread una rama **develop** desde **main**

5. Proteged la rama **main** para que se requieran de (n-1) aprobaciones, siendo n el nº de componentes del grupo

6. A partir de ahora, cada integrante del grupo trabajará de la siguiente manera:
   a. Trabajaremos en una rama con un nombre que represente al requisito (ej. exportacion, login, imagenes...)
   b. Una vez acabada la implementación haremos mezclaremos nuestra rama con **develop**. No borres la rama.
   c. Si todo ha ido bien, haremos un **pull request** de **develop** con **main**
   d. El resto de compañeros inspeccionarán los cambios y, si son correctos, darán su aprobación.
   e. Con todas las aprobaciones conseguidas, se mezclará **develop** con **main**. No borréis la rama **develop**